Agentic AI-Enabled Real-time Air Drawing Application

A Report submitted in partial fulfillment of the requirements
for the Degree of

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING

by

G. Geethanjali	**(2311CS010192)**
G. Madhu	**(2311CS010252)**
M. Sai Kumar	**(2311CS010455)**
K. Jagadeshwar	**(2311CS010279)**

Under the guidance of

Mr. Srikanth Dasari

Assistant Professor

Department of Computer Science and Engineering
School of Engineering

MALLA REDDY UNIVERSITY

Maisammaguda Dulapally Hyderabad Telangana 500100

2026

Agentic AI-Enabled Real-time Air Drawing Application

A Report submitted in partial fulfillment of the requirements
for the Degree of

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING

by

G. Geethanjali	**(2311CS010192)**
G. Madhu	**(2311CS010252)**
M. Sai Kumar	**(2311CS010455)**
K. Jagadeshwar	**(2311CS010279)**

Under the guidance of

Mr. Srikanth Dasari

Assistant Professor



Department of Computer Science and Engineering

School of Engineering

MALLA REDDY UNIVERSITY

Maisammaguda, Dulapally, Hyderabad, Telangana 500100

2026

II

MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Major Project report entitled **"Agentic AI-Enabled Real-time Air Drawing Application"** is submitted by **G. Geethanjali (2311CS010192), G. Madhu (2311CS010252), M. Sai Kumar (2311CS010455) and K.Jagadeshwar (2311CS010279**), to the Department of Computer Science and Engineering, Malla Reddy University, Hyderabad, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

The report presents the problem, literature survey, objectives, methodology, testing, implementation, results and conclusions of the project 2026

INTERNAL GUIDE

Mr. Srikanth Dasari

DEAN CSE

Dr. A.V.L.N. Sujith

External Examiner

MALLA REDDY UNIVERSITY
(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science and Engineering

DECLARATION

We hereby declare that the project report entitled "Inspired Evolutionary Optimisation with Dynamic Updates for Electric Vehicle Fleet Routing in Smart City Environments" has been prepared as part of our major project in the Department of Computer Science and Engineering, Malla Reddy University, Hyderabad, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology.

Published literature, the associated research manuscript, software resources and experimental records used in this work are acknowledged through citations and references. Prior submissions and individual contributions will be declared in accordance with the institution's academic integrity requirements.

Place Hyderabad

Date

G. Geethanjali	**(2311CS010192)**
G. Madhu	**(2311CS010252)**
M. Sai Kumar	**(2311CS010455)**
K. Jagadeshwar	**(2311CS010279)**

ACKNOWLEDGEMENT

We express our sincere gratitude to **Mr. Chamakura Malla Reddy**, Chairman, and **Dr V. S. K. Reddy**, Vice Chancellor, Malla Reddy University, for the academic environment and institutional support provided for this project.

We thank our project guide, **Mr. Srikanth Dasari, Associate Professor**, Department of Computer Science and Engineering, for guidance, constructive feedback and support throughout the work.

We thank **Dr. A.V.L.N. Sujith**, Dean CSE, School of Engineering, and the faculty of the department for their encouragement. We acknowledge the researchers whose published work provides the foundations for the routing, evolutionary search and adaptive-memory methods the foundation for the study, together with the contributors to the software and dataset resources used in the project.

We thank our families, friends and peers for their encouragement during the project.

G. Geethanjali **(2311CS010192)**

G. Madhu **(2311CS010252)**

M. Sai Kumar **(2311CS010455)**

K. Jagadeshwar **(2311CS010279)**

ABSTRACT

The proposed Real-Time Air Drawing Application is a touchless drawing system that allows users to draw on a virtual canvas using simple hand gestures captured through a webcam. The main aim of the system is to provide a natural and convenient way of interacting with a computer without using a mouse, keyboard, or touchscreen. OpenCV and MediaPipe are used for real-time video processing, hand detection, landmark tracking, and gesture recognition.

The system continuously tracks the user's hand and index finger to control the drawing process. Drawing starts when the index finger is extended and its movement is tracked on the screen to create smooth strokes. Drawing stops when the whole hand is closed, which helps prevent unwanted or accidental strokes. This simple gesture-based interaction makes the application easy to use and provides a more natural drawing experience.

A virtual toolbar is included to provide useful drawing controls such as color selection, erasing, clearing the canvas, undo, redo, and saving the drawing. These operations can be performed using hand gestures without physically touching the screen. The system efficiently stores and displays the detected drawing points to provide smooth and responsive output in real time.

An Agentic AI layer is also included to provide higher-level decision-making by interpreting meaningful gesture events and selecting the appropriate action. This makes the system more intelligent and provides a foundation for adding advanced gesture-based features in the future. Overall, the proposed application demonstrates the use of computer vision, hand tracking, and intelligent decision-making to create an interactive and contactless human-computer interaction system. It can be useful for education, presentations, creative activities, and other applications where touchless interaction is beneficial.

TABLE OF CONTENTS

DESCRIPTION	PAGE NUMBER
ABSTRACT	VI
Chapter 1 Introduction	1
1.1 Outline of Project	1
1.2 Problem Definition	2
Chapter 2 Literature Survey	3
2.1 Survey	3
2.2 Previous Studies and Existing System	4
Chapter 3 Project Objectives	7
3.1 Objective of the Project	7
3.2 Limitations of the Project	9
Chapter 4 Design And Methodology	10
4.1 Proposed System	10
4.2 Architecture	12
4.3 Modules	13
4.4 Methods and Algorithms	14
4.5 Structural Design	16
4.6 System Design	17
4.7 Data Flow in the System	18
4.8 UML Diagram	19
Chapter 5 Testing	24
Need of Testing	24
Chapter 6 Code	25
6.1 System Implementation and outputs	34

TABLE OF CONTENTS CONTINUED

DESCRIPTION	PAGE NUMBER
Chapter 7 Results And Discussion	**35**
7.1 Results	**35**
7.2 Discussion	**36**
7.3 Existing system vs Proposed System	**37**
Chapter 8 Conclusion	**38**
8.1 Project Conclusion	**38**
Future Scope	**39**
References	**40**

LIST OF FIGURES

FIG NO	FIG NAME	PAGE NO
Figure 4.1	Proposed System	10
Figure 4.2	Architecture Diagram	12
Figure 4.3	Data Flow Diagram	18
Figure 4.4	Use Case Diagram	19
Figure 4.5	Class Diagram	20
Figure 4.6	Sequence Diagram	21
Figure 4.7	Activity Diagram	22
Figure 4.8	Deployment Diagram	23
Figure 6.1.1	Virutual Canvas Manager Implementation	34

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
Table 2.1	Literature perspectives and evaluation requirements	6
Table 7.3	Existing system and proposed-system comparison	37

ABBREVIATIONS AND NOTATION

AI	Artificial Intelligence
Agentic AI	Agentic Artificial Intelligence
CV	Computer Vision
HCI	Human-Computer Interaction
OpenCV	Open Source Computer Vision Library
MP	MediaPipe
FPS	Frames Per Second
GUI	Graphical User Interface
UI	User Interface
API	Application Programming Interface
RGB	Red, Green, Blue
BGR	Blue, Green, Red
IDE	Integrated Development Environment
ML	Machine Learning
DL	Deep Learning
CPU	Central Processing Unit
RAM	Random Access Memory
USB	Universal Serial Bus
I/O	Input/Output
2D	Two-Dimensional

CHAPTER-1

INTRODUCTION

1.1 Outline of Project

The Real-Time Air Drawing Application is a touchless drawing system that allows users to draw on a virtual canvas using hand gestures. The main purpose of this project is to provide a simple and natural way of interacting with a computer without using a mouse, keyboard, or touchscreen. The system uses a webcam to capture the user's hand movements and converts them into drawing actions on the screen.

The application uses OpenCV for capturing and processing the webcam video and MediaPipe for detecting and tracking the hand landmarks. The index finger is mainly used to control the drawing movement. When the index finger is extended, the system tracks its movement and creates a corresponding stroke on the virtual canvas. When the whole hand is closed, the drawing operation is stopped. This helps the user control the drawing easily and reduces unwanted strokes.

The application also provides a virtual toolbar with different options such as color selection, eraser, clear, undo, redo, and save. These features allow users to manage their drawings without physically touching the computer. The system is designed to provide smooth and responsive drawing by continuously processing the detected hand movements and storing the required drawing points.

An Agentic AI component is included to provide an intelligent decision-making layer. It interprets meaningful gesture events and helps select the appropriate action based on the user's interaction. This makes the application more than a basic hand-tracking system and provides a foundation for developing more intelligent gesture-based interactions.

The proposed system can be useful in classrooms, presentations, demonstrations, creative activities, and other situations where touchless interaction is helpful. Overall, the project demonstrates how computer vision, hand tracking, and Agentic AI can be combined to create a simple, interactive, and contactless human-computer interaction system.

1.2 Problem Definition

Traditional drawing and computer interaction systems mainly depend on physical devices such as a mouse, keyboard, stylus, or touchscreen. These methods require direct physical interaction and may not always be convenient, especially during classroom teaching, presentations, demonstrations, or situations where touchless interaction is preferred. Using a mouse or other input devices can also make drawing and controlling on-screen content less natural.

The main problem addressed by this project is to develop a simple and effective system that allows users to draw and control basic drawing functions without physically touching the computer. The system should be able to recognize hand movements through a webcam, identify the user's drawing gestures, and convert them into actions on a virtual canvas.

The proposed Real-Time Air Drawing Application uses OpenCV and MediaPipe for real-time hand detection and tracking. The system uses the index finger to draw on the virtual canvas and stops drawing when the whole hand is closed. It also provides virtual controls for selecting colors, erasing, clearing, undoing, redoing, and saving drawings. An Agentic AI layer helps interpret meaningful gesture events and select the appropriate actions.

Therefore, the project aims to provide a simple, user-friendly, responsive, and touchless drawing system that enables natural human-computer interaction while reducing the need for traditional physical input devices.

CHAPTER-2

LITERATURE SURVEY

2.1 Survey

The literature survey focuses on the research areas related to touchless human-computer interaction, hand gesture recognition, computer vision, virtual drawing, and intelligent interaction. Its purpose is to understand the existing approaches used for detecting and tracking hand movements and to position the proposed Real-Time Air Drawing Application with respect to these techniques.

This is a focused review of technologies and approaches relevant to the proposed system rather than an exhaustive systematic literature review. OpenCV and MediaPipe are considered for real-time video processing and hand landmark detection, while gesture-based interaction methods provide the foundation for controlling drawing operations without physical input devices. The survey provides background on how hand movements can be converted into meaningful computer actions.

Existing air-drawing systems mainly use hand or fingertip movements to create strokes on a virtual canvas. These approaches demonstrate the usefulness of camera-based interaction, but they may provide limited drawing controls or depend on a small number of predefined gestures. The proposed system considers these limitations by including features such as color selection, erasing, clear, undo, redo, and saving drawings through a virtual toolbar.

The survey therefore supports the development of a system that combines webcam-based video processing, hand tracking, gesture recognition, virtual drawing controls, and Agentic AI. The proposed application aims to provide smooth, responsive, and natural touchless interaction while addressing practical requirements such as reducing accidental strokes and providing basic drawing management features.

2.2 Previous Studies and Existing System

Previous studies on touchless human-computer interaction have explored the use of computer vision and hand gestures to control computer applications without physical input devices. Many existing systems use a webcam to capture hand movements and identify specific gestures for performing actions. These approaches have shown that hand tracking can provide a simple and natural alternative to traditional devices such as a mouse, keyboard, or touchscreen.

Existing air-drawing systems generally use the movement of the index finger or another hand landmark to draw on a virtual canvas. The detected fingertip position is continuously tracked and converted into drawing points. Some systems also provide basic features such as changing colors, erasing, or clearing the canvas. However, many of these systems are mainly focused on drawing and provide limited options for managing the drawing or controlling other operations through gestures.

Computer vision libraries such as OpenCV are commonly used for webcam video capture and image processing, while MediaPipe provides hand landmark detection and tracking. These technologies make real-time gesture recognition possible without requiring specialized hardware. However, detecting hand movement alone does not always provide intelligent interpretation of user intentions. Simple gesture-based systems may also produce unwanted strokes when the user moves the hand without intending to draw.

The existing system generally depends on predefined gestures and direct gesture-to-action mapping. It may not include an intelligent layer for interpreting a sequence or meaning of user actions. The proposed system improves upon this approach by using the index finger for drawing and stopping the drawing operation when the whole hand is closed. It also provides a virtual toolbar for color selection, erasing, clearing, undo, redo, and saving.

In addition, the proposed system introduces an Agentic AI layer to provide higher-level decision-making based on meaningful gesture events. This combination of computer vision, hand tracking, virtual controls, and intelligent decision-making aims to provide a more complete, responsive, and user-friendly touchless drawing experience compared with basic existing air-drawing systems.

2.3 Previous Studies and Existing System continued

Previous studies on hand gesture-based interaction have explored different approaches for controlling computer applications using camera input. Hand gesture recognition techniques are used to identify finger movements and convert them into commands. These studies provide the foundation for touchless interaction, but their methods and supported operations vary depending on the application.

OpenCV-based systems provide real-time webcam processing and image manipulation, while MediaPipe-based approaches provide efficient hand landmark detection and tracking. These technologies are commonly used in applications such as virtual drawing, presentation control, and gesture-based interfaces. However, some existing systems focus mainly on detecting gestures and do not provide a complete drawing environment with multiple editing and management features.

Existing air-drawing approaches generally use fingertip movement to generate drawing strokes on a virtual canvas. Some systems support basic operations such as color selection, erasing, and clearing, but may have limited mechanisms for preventing accidental drawing or managing previous actions. The proposed system addresses this by using the extended index finger for drawing and stopping the drawing when the whole hand is closed.

Recent AI-based interaction systems also demonstrate the use of intelligent decision-making for understanding user actions. The proposed system extends the basic gesture-based approach by introducing an Agentic AI layer that interprets meaningful gesture events and selects appropriate actions. Therefore, the existing studies provide the technical foundation for the proposed application, while the present system combines hand tracking, virtual drawing controls, and intelligent decision-making into a single touchless drawing environment.

2.2 Previous Studies and Existing System comparison

The selected references address different parts of the research problem. Some motivate search representation, others address energy constraints, and others explain adaptation or local improvement. A comparison matrix is useful when it preserves these differences rather than reducing every paper to a single performance ranking.

Table 2.1 Literature perspectives and evaluation requirements

Research area	References	Implication
Hand detection and tracking	[1]	Accurate real-time hand landmark detection is required for reliable gesture interaction.
Air writing and drawing	[2, 3]	Supports drawing using finger movements on screen
Hand gesture recognition	[4,5,6]	Helps identify and classify different hand gestures
Virtual drawing interaction	[2,3,7]	Provides touchless drawing on a virtual canvas smoothly
Touchless interaction	[4,5,6]	Enables natural interaction without physical input devices
Intelligent interaction	[7]	Improves gesture-based decision-making

Zhang and co-authors [1] introduced MediaPipe Hands for real-time hand tracking. Nitin Kumar and co-authors [2] and Heena and Sandeep Ranjan [3] explored air writing using MediaPipe and OpenCV. Oudah and co-authors [4] and Sarma and Bhuyan [5] studied vision-based hand gesture recognition for human-computer interaction. These studies provide the basic foundation for touchless drawing and gesture-based interaction.

The existing systems mainly focus on hand tracking, gesture recognition, and basic air drawing. The proposed system extends these ideas by using the index finger for drawing and stopping when the whole hand is closed. It also provides color, erase, clear, undo, redo, and save options, along with an Agentic AI layer for interpreting meaningful gestures and selecting suitable actions.

CHAPTER-3

PROJECT OBJECTIVES

3.1 Objective of the Project

The main objective of the Real-Time Air Drawing Application is to develop a simple and user-friendly touchless drawing system that allows users to draw on a virtual canvas using hand gestures captured through a webcam. The system aims to provide a natural way of interacting with a computer without using a mouse, keyboard, or touchscreen.

The project aims to use OpenCV and MediaPipe to capture video, detect hand landmarks, and track finger movements in real time. Drawing is performed by extending and moving the index finger, while drawing stops when the whole hand is closed. This helps provide better control and reduces unwanted strokes.

Another objective is to provide useful drawing features such as color selection, eraser, clear, undo, redo, and save through a virtual toolbar. An Agentic AI layer is included to interpret meaningful gesture events and select suitable actions based on user interaction.

The project also aims to provide smooth and responsive drawing with simple gesture controls. The application can be useful in classrooms, presentations, demonstrations, and creative activities where touchless interaction makes computer use more convenient.

3.1.1 Research Questions

The study is organised around six research questions that examine the effectiveness of the proposed Real-Time Air Drawing Application. Each question focuses on a key aspect of the system, including hand tracking, gesture recognition, drawing control, and intelligent interaction. These questions guide the development and evaluation of the application.

RQ1: How accurately can OpenCV and MediaPipe detect and track hand movements in real time?

RQ2: How effectively can index finger movements be converted into smooth and continuous strokes on a virtual canvas?

RQ3: How reliably can the system detect a closed hand gesture to stop drawing and reduce unwanted strokes?

RQ4: How effectively can users control drawing tools such as color selection, eraser, clear, undo, redo, and save using hand gestures?

RQ5: How can Agentic AI interpret meaningful gesture events and select appropriate actions to improve user interaction?

RQ6: How well does the proposed application provide a smooth, responsive, and user-friendly touchless drawing experience?

These research questions provide a basis for evaluating the functionality and usability of the proposed system. The results can help identify the strengths and limitations of the application and determine areas for future improvement.

3.2 Limitations of the Project

The Real-Time Air Drawing Application has certain limitations that may affect its performance and usability. Since the system uses a webcam to detect and track hand movements, its accuracy depends on camera quality, lighting conditions, and the visibility of the user's hand. Poor lighting or a low-quality webcam may affect hand detection and drawing accuracy.

The application mainly depends on predefined hand gestures, such as extending the index finger to draw and closing the whole hand to stop drawing. Unclear hand movements or incorrect gesture detection may cause unwanted strokes or interruptions. The smoothness of drawing may also depend on system performance and the speed of video processing.

The system requires a working webcam and sufficient computing resources to process hand movements in real time. Hand tracking may become less accurate when the hand moves too quickly, is partially hidden, or moves outside the camera's view. In addition, the Agentic AI component is limited to the gesture interpretation and actions implemented in the application.

The application is primarily designed for basic drawing and touchless interaction. It may not provide the advanced editing features, precision, and pressure sensitivity available in professional digital drawing tools. Therefore, further improvements are needed to enhance accuracy, support more gestures, and provide a better drawing experience.

.

CHAPTER-4

DESIGN AND METHODOLOGY

4.1 Proposed System

The proposed system combines real-time video processing, hand landmark tracking, gesture recognition, and intelligent decision-making to provide a touchless drawing experience. OpenCV captures and processes webcam video, while MediaPipe detects and tracks hand movements. The index finger is used to draw on the virtual canvas, and drawing stops when the whole hand is closed.

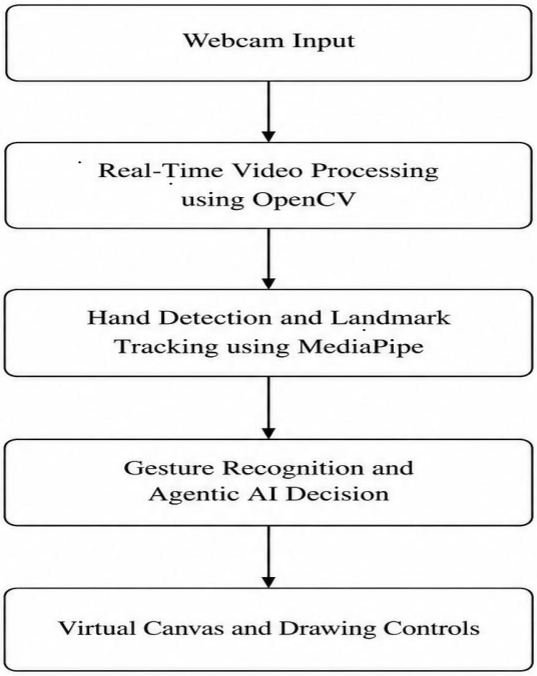


Fig 4.1 Proposed System

The system continuously processes webcam frames and tracks the required hand landmarks to identify drawing movements. The detected finger positions are converted into coordinates on the virtual canvas to create strokes. Gesture recognition helps control drawing actions and reduces unwanted marks. The drawing controls allow users to modify and manage their work easily.

4.1.1 Problem Formulation

The main problem addressed in this project is to develop a real-time drawing application that allows users to draw on a virtual canvas without touching the computer. Traditional drawing methods depend on devices such as a mouse, keyboard, or touchscreen, which may not always be convenient for presentations, classroom activities, and demonstrations.

The proposed system uses a webcam to capture hand movements and OpenCV to process video frames. MediaPipe detects and tracks hand landmarks to identify the index finger's position. When the index finger is extended, its movement is converted into drawing strokes on the virtual canvas. When the whole hand is closed, drawing stops to prevent unwanted marks.

The system also needs to provide simple controls for selecting colors, erasing, clearing, undoing, redoing, and saving drawings. An Agentic AI layer is included to interpret meaningful gesture events and select suitable actions. The main challenge is to ensure accurate gesture detection, smooth drawing, quick response, and reliable control of the virtual tools.

Therefore, the problem is formulated as designing a user-friendly and responsive touchless drawing system that combines real-time hand tracking, gesture-based control, virtual drawing tools, and intelligent decision-making to support natural human-computer interaction.

4.2 Architecture

The architecture of the Real-Time Air Drawing Application consists of webcam input, video processing, hand landmark tracking, gesture recognition, intelligent decision-making, and virtual drawing controls. Each component performs a specific task to convert hand movements into drawing actions. OpenCV and MediaPipe are used to process video frames and track hand movements, while the Agentic AI layer supports gesture interpretation and action selection.

.

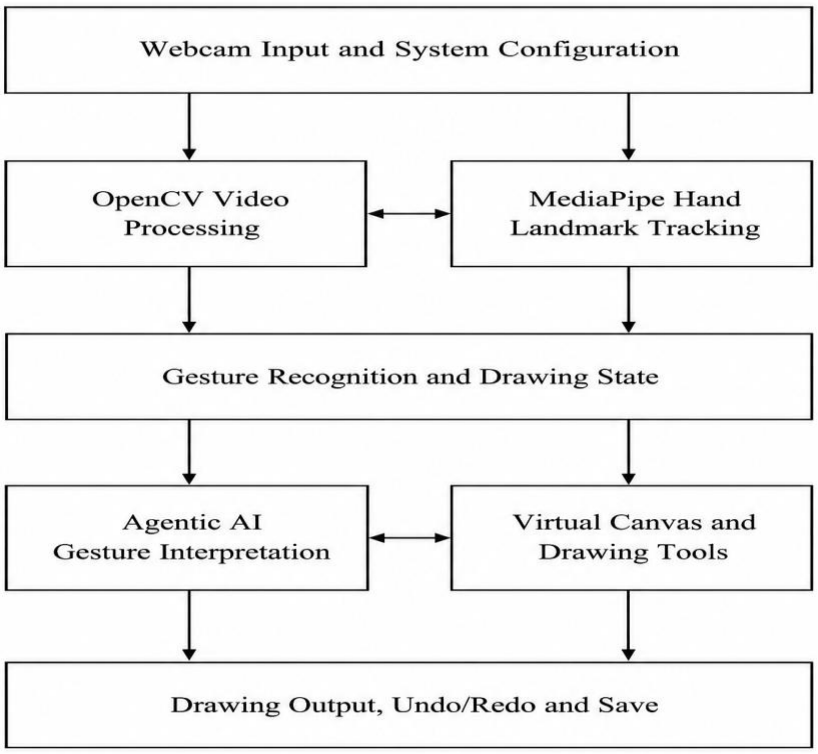


Fig 4.2 Architecture Diagram

The virtual canvas displays the drawing based on the detected finger movements. Users can select colors, erase strokes, clear the canvas, undo or redo actions, and save their drawings. These components work together to provide a smooth and responsive touchless drawing experience. The architecture is designed to make the application simple, interactive, and suitable for educational activities, presentations, and demonstrations.

4.3 Modules

The Real-Time Air Drawing Application is divided into several modules, with each module performing a specific function to ensure smooth and effective operation.

The webcam input module captures live video of the user's hand. The video processing module uses OpenCV to process the captured frames and prepare them for hand tracking. The hand detection and tracking module uses MediaPipe to identify hand landmarks and track finger movements in real time.

The gesture recognition module identifies the user's hand gestures. When the index finger is extended, the system allows drawing on the virtual canvas. When the whole hand is closed, drawing stops. This helps control the drawing process and reduce unwanted strokes.

The virtual canvas module converts finger movements into drawing strokes and displays them on the screen. The drawing tools module provides features such as color selection, eraser, clear, undo, redo, and save. These controls help users manage their drawings easily.

The Agentic AI module interprets meaningful gesture events and helps select suitable actions based on the user's interaction. Finally, the output module displays the completed drawing and supports saving the work for future use. Together, these modules provide a simple, responsive, and user-friendly touchless drawing application.

4.4 Methods and Algorithms

The Real-Time Air Drawing Application uses computer vision, hand tracking, gesture recognition, and intelligent decision-making to enable touchless drawing. These methods work together to detect hand movements and convert them into drawing actions on a virtual canvas.

The video capture and processing method uses OpenCV to access the webcam and process video frames continuously. MediaPipe is used for hand detection and landmark tracking. It identifies key points on the hand, including the index finger, and provides their positions for controlling drawing movements.

The gesture recognition method determines the user's intended action based on hand positions. When the index finger is extended, the system tracks its movement and draws corresponding strokes on the canvas. When the whole hand is closed, drawing stops. This method helps provide better control and reduces accidental strokes.

The coordinate mapping algorithm converts the detected finger positions from the webcam frame into corresponding positions on the virtual canvas. The system stores successive drawing points and connects them to create continuous strokes. This helps produce smooth drawing output during real-time interaction.

The Agentic AI method interprets meaningful gesture events and helps select appropriate actions, such as drawing or activating a drawing control. The virtual toolbar method supports color selection, erasing, clearing, undoing, redoing, and saving drawings. These actions allow users to manage their work through the application's gesture-based controls.

Finally, the drawing rendering and storage method displays the strokes on the virtual canvas and maintains the information required for undo and redo operations. The drawing can also be saved for future use. Together, these methods and algorithms provide an interactive, responsive, and user-friendly touchless drawing experience.

.

4.4.1 Dynamic Recovery and Memory

Dynamic recovery and memory in the Real-Time Air Drawing Application help maintain smooth interaction and preserve drawing progress during user operations. The system tracks hand movements continuously and updates the virtual canvas according to the detected gestures. If the hand is temporarily not detected, the system can pause drawing until tracking resumes, helping to avoid unwanted strokes.

The application maintains drawing points and stroke information so that the user's work can be managed efficiently. The undo and redo features use stored drawing states to remove or restore recent actions. This allows users to correct mistakes without starting the drawing again. The clear option removes the current drawing when required.

The system also maintains the information needed to manage drawing controls, including the selected color and eraser mode. When hand tracking resumes, the application can continue responding to recognized gestures. The Agentic AI layer can interpret meaningful gesture events and help select suitable actions.

Overall, dynamic recovery and memory improve the reliability and usability of the application by supporting drawing corrections, maintaining relevant drawing information, and reducing interruptions during touchless interaction.

4.5 Structural Design

The structural design of the Real-Time Air Drawing Application describes how the different components are organised and connected to perform touchless drawing. The system follows a modular structure in which each component has a specific responsibility, making the application easier to understand, develop, and maintain.

The webcam acts as the input source and captures the user's hand movements. OpenCV processes the video frames, while MediaPipe detects and tracks hand landmarks. The gesture recognition component analyses the detected hand positions to identify drawing and stopping actions. The recognized gestures are then used to control the virtual canvas.

The virtual canvas module displays the drawing created through finger movements. The drawing tools module manages color selection, eraser, clear, undo, redo, and save operations. The Agentic AI component provides an additional decision-making layer to interpret meaningful gesture events and select suitable actions.

All these components work together to provide real-time drawing and interactive controls. The modular structure also makes it easier to improve individual components, add new gestures, and introduce additional drawing features in the future. This design aims to provide a simple, responsive, and user-friendly touchless drawing system.

4.6 System Design

The system design of the Real-Time Air Drawing Application explains how the different components work together to provide touchless drawing. The system is designed to capture hand movements, recognize gestures, and convert them into actions on a virtual canvas. It combines computer vision, hand tracking, gesture recognition, and intelligent decision-making to provide a simple and interactive drawing experience.

The system begins by capturing live video through a webcam. OpenCV processes the video frames, and MediaPipe detects and tracks hand landmarks. The detected finger positions are analysed to identify the user's drawing gestures. When the index finger is extended, the system draws on the virtual canvas by following its movement. When the whole hand is closed, drawing stops.

The virtual canvas displays the strokes created by the user and provides drawing controls such as color selection, eraser, clear, undo, redo, and save. The Agentic AI layer interprets meaningful gesture events and helps select the appropriate actions. The system also maintains drawing information to support corrections and manage drawing operations.

The overall design focuses on accuracy, responsiveness, simplicity, and ease of use. By combining these components, the application provides a touchless drawing environment that can be used in classrooms, presentations, demonstrations, and other situations where natural computer interaction is useful.

4.7 Data Flow in the System

Data enters the system through the webcam as live video frames. OpenCV processes the frames, and MediaPipe detects and tracks hand landmarks. The gesture recognition module identifies the user's hand movements, while the Agentic AI layer interprets meaningful gesture events and helps select the appropriate actions. The system then updates the virtual canvas based on the recognized gestures.

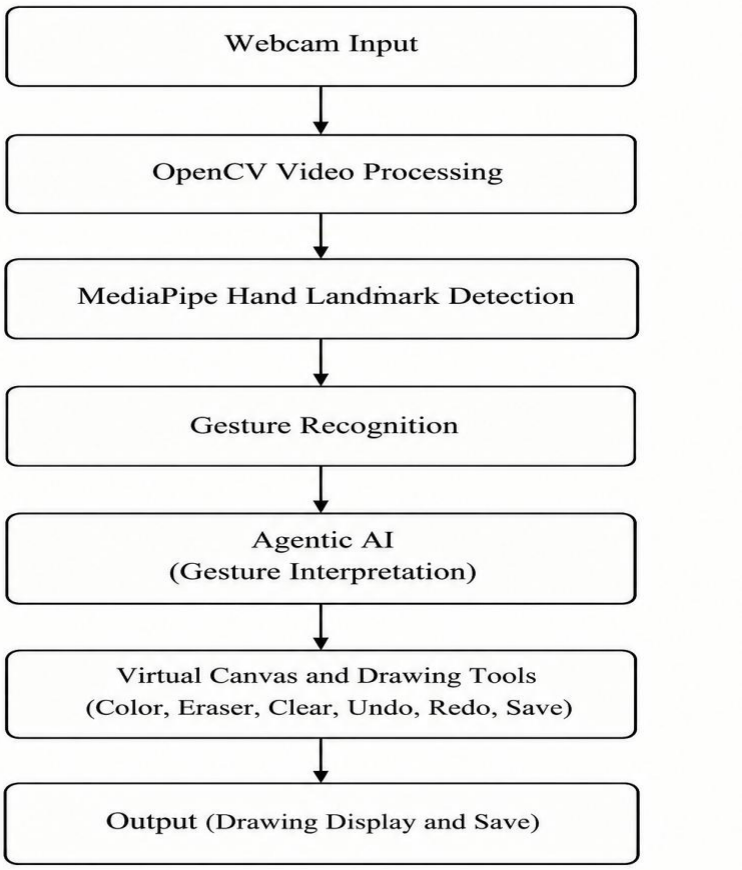


Fig 4.3 Data Flow Diagram

The system processes hand movements continuously to provide real-time drawing. When the index finger is extended, its movement creates strokes on the canvas, and drawing stops when the whole hand is closed. The application maintains drawing information to support color selection, erasing, clearing, undo, redo, and save operations. The final output is displayed on the virtual canvas, providing a smooth and interactive touchless drawing experience.

4.8 UML Diagram

4.8.1 Use Case Diagram

The use case diagram represents the research workflow. The researcher selects inputs and runs experiments; a reviewer can inspect records and execute the verification or analysis commands. These roles are interactions with the software, not claims about an implemented access-control system.

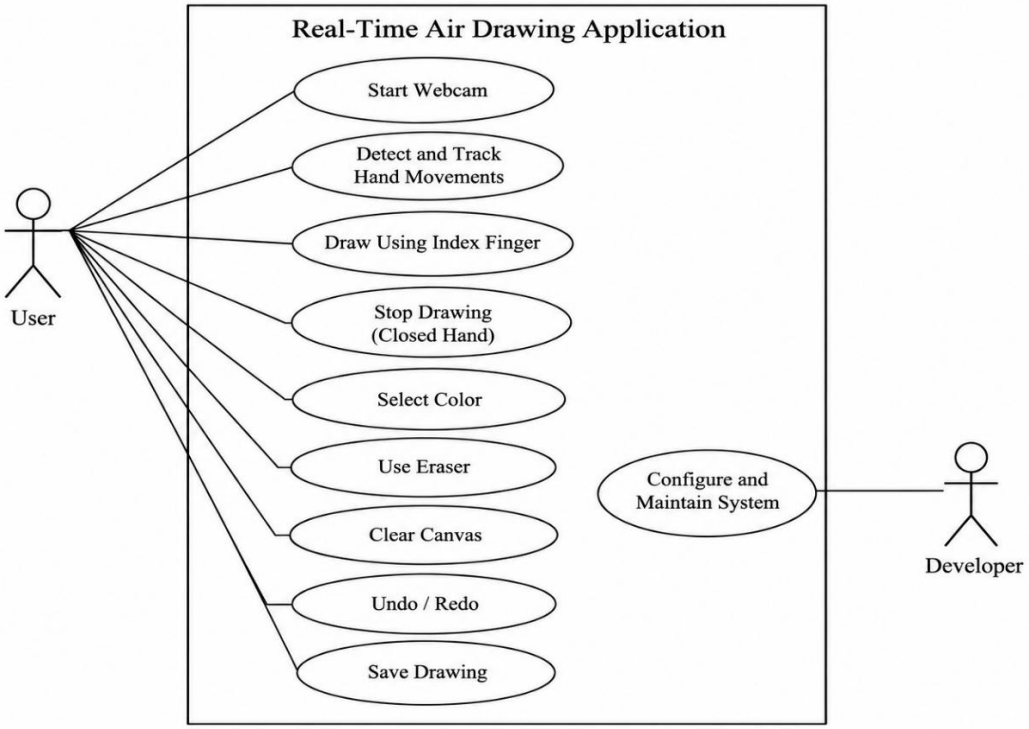


Fig 4.4 Use Case Diagram

The diagram shows the main functions provided by the application. The system processes hand gestures and performs the corresponding actions on the virtual canvas. These interactions provide a simple and convenient way to draw without using physical input devices. The developer is responsible for configuring and maintaining the application.

4.8.2 Class Diagram

The class diagram represents the main components of the Real-Time Air Drawing Application and their relationships. The WebcamInput class captures video frames, while OpenCVProcessor processes the frames and HandTracker detects and tracks hand landmarks. The GestureRecognizer identifies hand gestures and determines whether the user wants to draw or stop drawing.

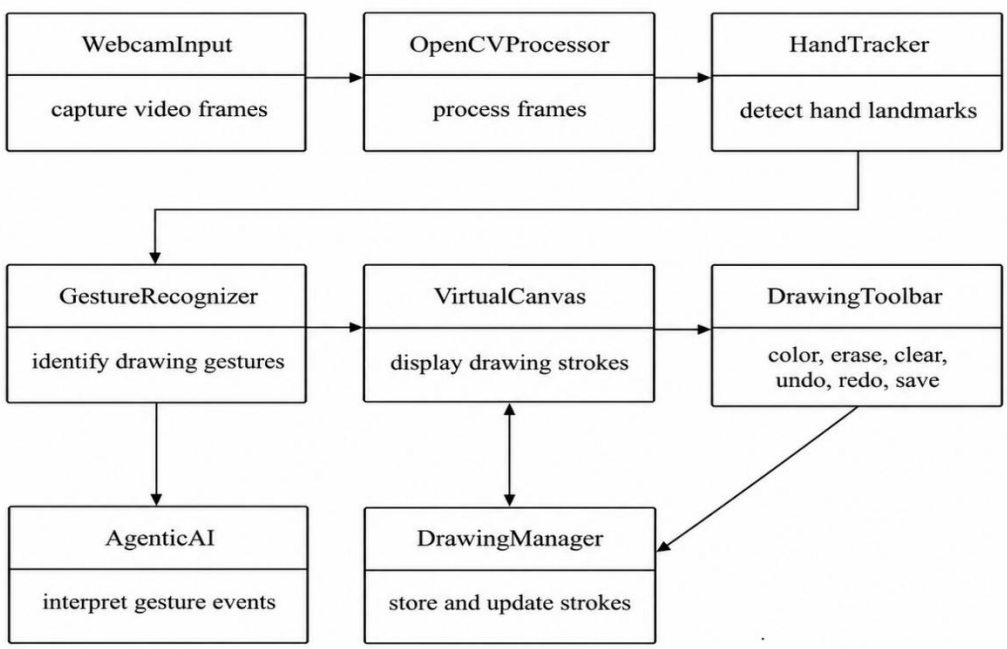


Fig 4.5 Class Diagram

These classes work together to provide real-time hand tracking, gesture-based drawing, and efficient canvas management. The modular design makes the application easier to maintain and allows new features to be added in the future.

4.8.3 Sequence Diagram

The sequence diagram illustrates the sequence of interactions between the user, webcam, hand tracking module, gesture recognition module, virtual canvas, and drawing manager. It explains how the system processes hand movements and converts them into drawing actions in real time.

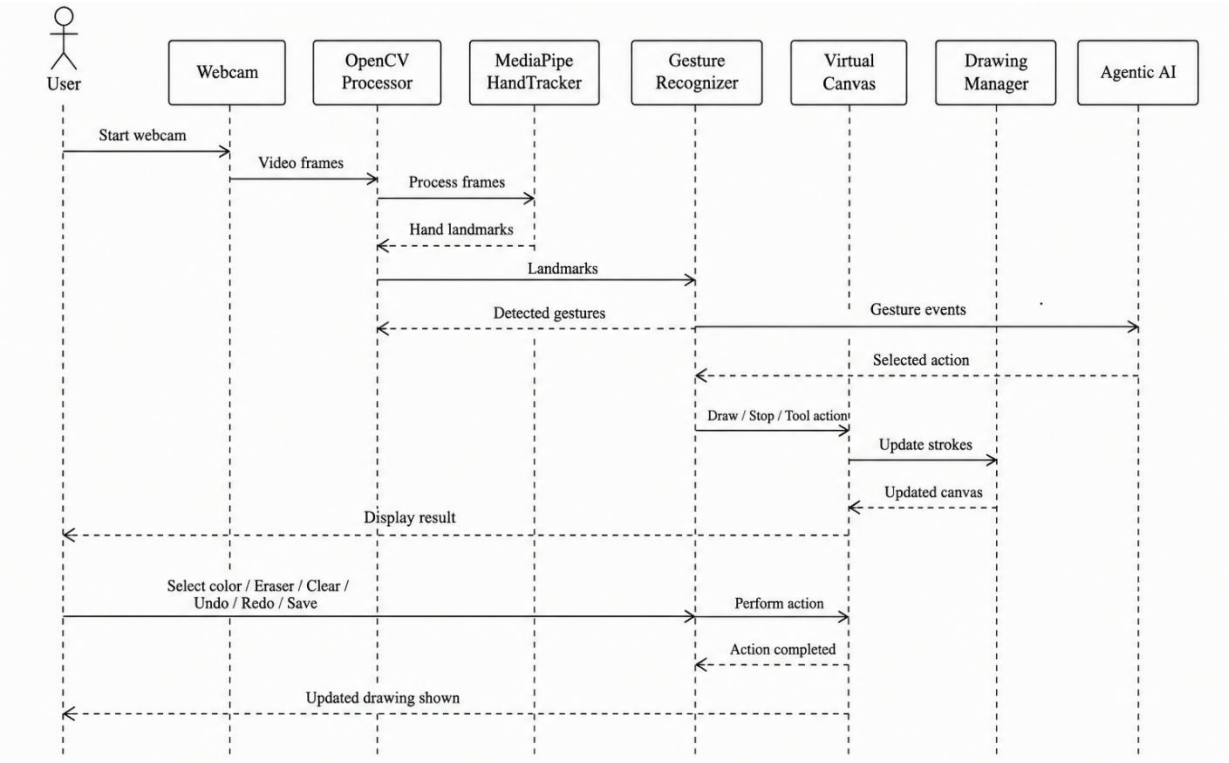


Fig 4.6 Sequence Diagram

The drawing manager stores and updates the strokes, while the virtual toolbar handles options such as color selection, eraser, clear, undo, redo, and save. The Agentic AI layer interprets meaningful gesture events and helps select suitable actions. These interactions work together to provide a smooth and responsive touchless drawing experience.

4.8.4 Activity Diagram

The activity diagram illustrates the sequence of activities performed in the Real-Time Air Drawing Application. The process begins by starting the webcam and capturing live video frames. OpenCV processes the frames, and MediaPipe detects and tracks the user's hand landmarks. If the hand is not detected, the system waits until the hand becomes visible.

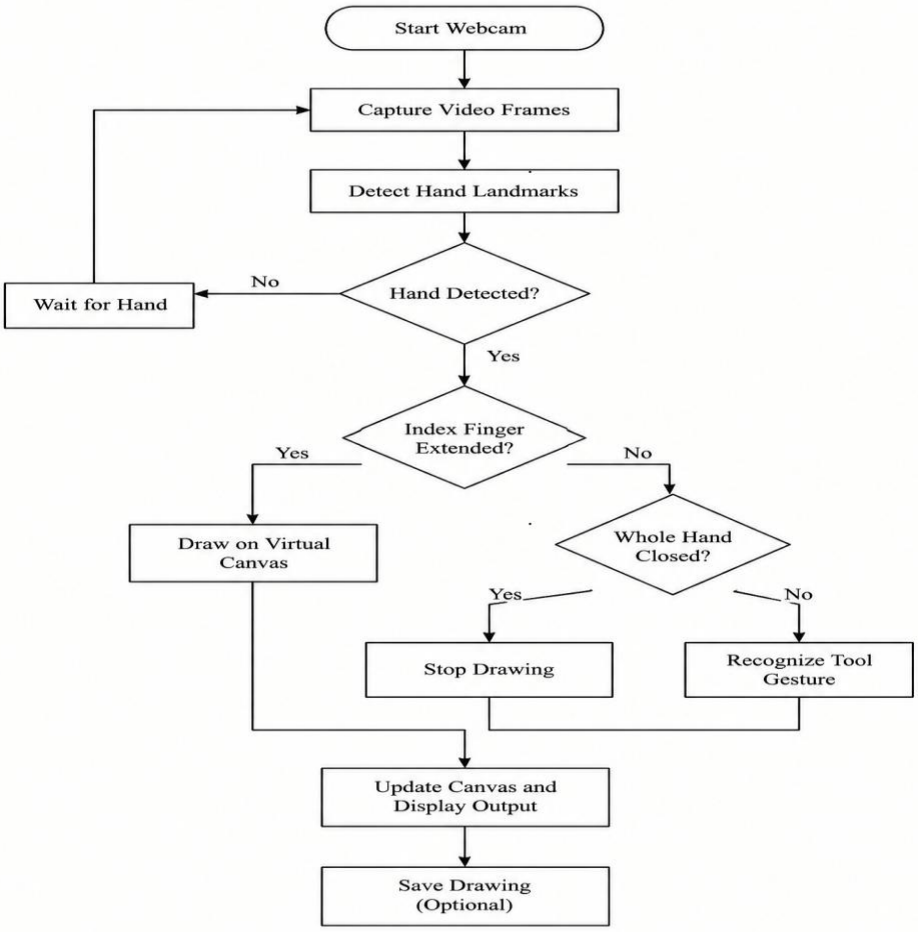


Fig 4.7 Activity Diagram

The system updates the virtual canvas based on the detected gestures and displays the output in real time. Users can also clear the canvas, undo or redo actions, and save their drawings. This sequence of activities provides a simple and interactive touchless drawing experience.

4.8.5 Deployment Diagram

The deployment diagram illustrates the deployment of the Real-Time Air Drawing Application on a local computing system. The diagram separates the webcam input, hand tracking and gesture processing, virtual drawing canvas, and local storage components. The system uses OpenCV for video capture and image processing, MediaPipe for hand landmark detection, and an Agentic AI component for interpreting gesture events and selecting appropriate actions. The application runs locally and does not require cloud services..

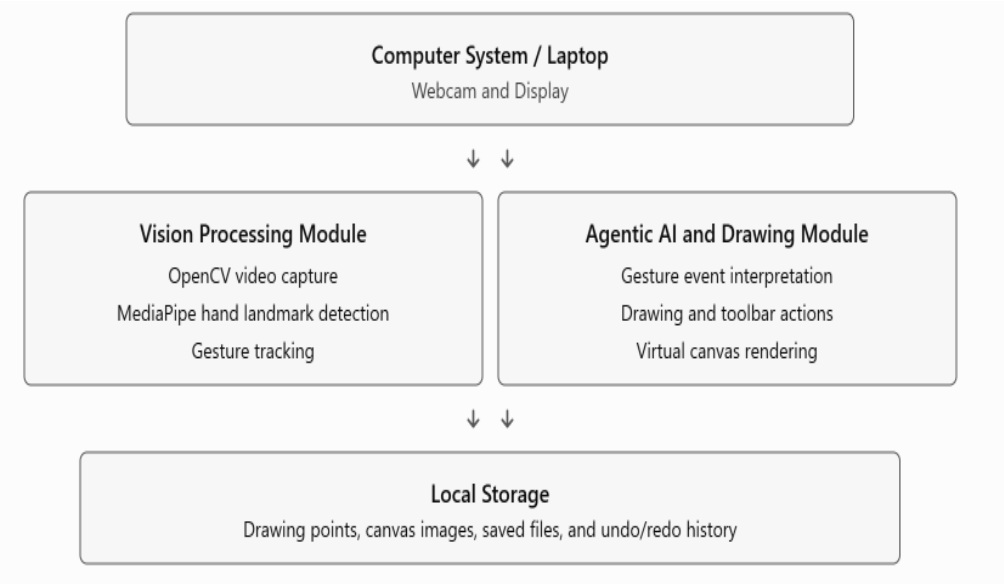


Fig 4.8 Deployment Diagram

The webcam captures live hand movements, which are processed by the vision module to identify hand landmarks and gestures. The Agentic AI component interprets recognized gesture events and selects the appropriate drawing operation, such as drawing, erasing, clearing, undoing, or redoing. The resulting strokes are displayed on the virtual canvas, while drawing data and saved images are managed through local storage.

CHAPTER-5

TESTING

Need of Testing

Testing is an essential phase in the development of the Real-Time Air Drawing Application because the system's effectiveness depends on the accurate detection of hand gestures and the correct execution of drawing actions. A drawing operation that appears successful may still produce unwanted strokes if hand landmarks are detected incorrectly or gesture transitions are not handled properly.

The existing testing process covers webcam initialization, hand landmark detection, gesture recognition, drawing operations, and toolbar functionality. These components must be verified individually to ensure that the application performs reliably under different operating conditions. Small controlled test cases allow individual features to be checked without treating a successful demonstration as proof of overall correctness.

Functional tests verify that index-finger movements produce corresponding strokes, closing the hand stops drawing, and toolbar options such as color selection, eraser, clear, undo, redo, and save perform their intended operations. Integration tests check whether OpenCV, MediaPipe, the virtual canvas, and the Agentic AI component exchange information correctly and execute the appropriate actions in response to recognized gestures.

CHAPTER-6 CODE

Source Code route evaluation

The Real-Time Air Drawing Application processes webcam input to detect hand movements and convert them into drawing actions on a virtual canvas. The system uses OpenCV for capturing and processing video frames and MediaPipe for detecting and tracking hand landmarks. The index finger is used to control the drawing position, while specific hand gestures determine whether drawing should continue or stop.

main.py

```
import cv2
import mediapipe as mp
import numpy as np
hands = mp.solutions.hands.Hands(max_num_hands=1)
draw = mp.solutions.drawing_utils
cap = cv2.VideoCapture(0)
canvas = None
px, py = 0, 0
while True:
    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.flip(frame, 1)
    h, w, _ = frame.shape
    if canvas is None:
        canvas = np.zeros_like(frame)
    result = hands.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
    if result.multi_hand_landmarks:
        for hand in result.multi_hand_landmarks:
            draw.draw_landmarks(frame, hand, mp.solutions.hands.HAND_CONNECTIONS)
            tip = hand.landmark[8]
            x, y = int(tip.x * w), int(tip.y * h)
```

```
            if tip.y < hand.landmark[6].y:
                if px and py:
                    cv2.line(canvas, (px, py), (x, y), (0, 0, 255), 5)
                px, py = x, y
            else:
                px, py = 0, 0
    else:
        px, py = 0, 0
    output = cv2.add(frame, canvas)
    cv2.imshow("Air Drawing", output)
    key = cv2.waitKey(1) & 0xFF
    if key == ord('c'):
        canvas[:] = 0
    elif key == ord('s'):
        cv2.imwrite("drawing.png", output)
    elif key == ord('q'):
        break
cap.release()
cv2.destroyAllWindows().
```

ai interface.py

```
class AICommandDispatcher:
    def __init__(self, canvas):
        self.canvas = canvas
    def execute_command(self, command):
        if not command:
            return "Please enter a command"
        command = command.lower()
        if "blue" in command:
            self.canvas.set_color("BLUE")
            return "Color changed to Blue"
        elif "green" in command:
       self.canvas.set_color("GREEN")
            return "Color changed to Green"
```

```
        elif "red" in command:
            self.canvas.set_color("RED")
            return "Color changed to Red"
        elif "clear" in command:
            self.canvas.clear()
            return "Canvas cleared"
        elif "save" in command:
            self.canvas.save()
            return "Drawing saved"
        elif "big" in command:
            self.canvas.set_brush_size("LARGE")
            return "Brush size increased"
        elif "small" in command:
            self.canvas.set_brush_size("SMALL")
            return "Brush size decreased"
        elif "erase" in command:
            self.canvas.show_toast("Close fist to erase")
            return "Eraser activated"
        else:
            return "Command not recognized"
```

drawing.py

```
import os
import time
import cv2
import numpy as np
class CanvasManager:
    COLORS = {
        "BLUE": (255, 110, 20),
        "GREEN": (40, 210, 50),
        "RED": (30, 40, 240),
        "BLACK": (25, 25, 25)
    }
    BRUSH_SIZES = {
```

```
    "SMALL": 4,
    "MEDIUM": 8,
    "LARGE": 16
}
def __init__(self, width=640, height=480):
    self.width = width
    self.height = height
    self.canvas_color = np.zeros((height, width, 3), dtype=np.uint8)
    self.canvas_mask = np.zeros((height, width), dtype=np.uint8)
    self.current_color = self.COLORS["BLUE"]
    self.brush_size = 8
    self.prev_point = None
    self.eraser_radius = 35
    self.toast_message = ""
    self.toast_expiry = 0
def resize(self, width, height):
    if self.width != width or self.height != height:
        self.width = width
        self.height = height
        self.canvas_color = np.zeros((height, width, 3), dtype=np.uint8)
        self.canvas_mask = np.zeros((height, width), dtype=np.uint8)
        self.prev_point = None
def set_color(self, color_name):
    color_name = color_name.upper()
    if color_name in self.COLORS:
        self.current_color = self.COLORS[color_name]
        self.show_toast("Color: " + color_name)

def set_brush_size(self, size):
    if isinstance(size, str):
        size = size.upper()
        if size in self.BRUSH_SIZES:
            self.brush_size = self.BRUSH_SIZES[size]
```

```
    elif isinstance(size, int) and 2 <= size <= 50:
        self.brush_size = size
    self.show_toast("Brush Size: " + str(self.brush_size))
def cycle_brush_size(self):
    sizes = list(self.BRUSH_SIZES.values())
    current = sizes.index(self.brush_size) if self.brush_size in sizes else 1
    self.brush_size = sizes[(current + 1) % len(sizes)]
    self.show_toast("Brush Size: " + str(self.brush_size))
def draw_stroke(self, point):
    if point is None:
        self.prev_point = None
        return
    x, y = point
    if self.prev_point is None:
        cv2.circle(self.canvas_color, (x, y), self.brush_size // 2,
                   self.current_color, -1)
        cv2.circle(self.canvas_mask, (x, y), self.brush_size // 2, 255, -1)
    else:
        cv2.line(self.canvas_color, self.prev_point, (x, y),
                 self.current_color, self.brush_size)
        cv2.line(self.canvas_mask, self.prev_point, (x, y),
                 255, self.brush_size)
    self.prev_point = (x, y)
def erase_at(self, point, radius=None):
    if point is None:
        self.prev_point = None
        return
    x, y = point
    r = radius if radius is not None else self.eraser_radius
    cv2.circle(self.canvas_color, (x, y), r, (0, 0, 0), -1)
    cv2.circle(self.canvas_mask, (x, y), r, 0, -1)
    self.prev_point = None
def clear(self):
```

```
        self.canvas_color.fill(0)
        self.canvas_mask.fill(0)
        self.prev_point = None
        self.show_toast("Canvas Cleared")
    def save(self, save_dir="saved_drawings"):
        os.makedirs(save_dir, exist_ok=True)
        filename = "drawing_" + time.strftime("%Y%m%d_%H%M%S") + ".png"
        filepath = os.path.join(save_dir, filename)
        image = np.full((self.height, self.width, 3), 255, dtype=np.uint8)
        image[self.canvas_mask > 0] = self.canvas_color[self.canvas_mask > 0]
        cv2.imwrite(filepath, image)
        self.show_toast("Drawing Saved")
        return filepath
    def reset_stroke(self):
        self.prev_point = None
    def blend(self, frame):
        output = frame.copy()
        mask = self.canvas_mask > 0
        output[mask] = self.canvas_color[mask]
        return output
    def show_toast(self, message, duration=2):
        self.toast_message = message
        self.toast_expiry = time.time() + duration
    def draw_toast_overlay(self, frame):
    if time.time() < self.toast_expiry and self.toast_message:
   cv2.putText(
      frame,
    self.toast_message,
    (20, self.height - 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (255, 255, 255),
```

handtracking.py

```
import os
import urllib.request
import cv2
import mediapipe as mp
class HandTracker:
    def __init__(self, max_num_hands=1, min_detection_confidence=0.7,
min_tracking_confidence=0.6):
        self.max_num_hands = max_num_hands
        self.min_detection_confidence = min_detection_confidence
        self.min_tracking_confidence = min_tracking_confidence
        self.use_legacy_solutions = hasattr(mp, "solutions") and hasattr(mp.solutions, "hands")
        if self.use_legacy_solutions:
            self.mp_hands = mp.solutions.hands
            self.detector = self.mp_hands.Hands(
                static_image_mode=False,
            max_num_hands=max_num_hands,
    min_detection_confidence=min_detection_confidence,
    min_tracking_confidence=min_tracking_confidence
)
self.mp_draw = mp.solutions.drawing_utils
        else:
            from mediapipe.tasks import python as mp_python
            from mediapipe.tasks.python import vision as mp_vision
            model_filename = "hand_landmarker.task"
            model_path = os.path.join(os.path.dirname(os.path.abspath(__file__)),
model_filename)
            if not os.path.exists(model_path):
                model_url = "https://storage.googleapis.com/mediapipe-
models/hand_landmarker/hand_landmarker/float16/latest/hand_landmarker.task"
                urllib.request.urlretrieve(model_url, model_path)
base_options = mp_python.BaseOptions(model_asset_path=model_path)
options = mp_vision.HandLandmarkerOptions(
```

```
            base_options=base_options,
            running_mode=mp_vision.RunningMode.IMAGE,
            num_hands=max_num_hands,
            min_hand_detection_confidence=min_detection_confidence,
            min_hand_presence_confidence=min_tracking_confidence,
            min_tracking_confidence=min_tracking_confidence
        )
        self.detector = mp_vision.HandLandmarker.create_from_options(options)
    self.CONNECTIONS = [
        (0, 1), (1, 2), (2, 3), (3, 4),
        (0, 5), (5, 6), (6, 7), (7, 8),
        (5, 9), (9, 10), (10, 11), (11, 12),
        (9, 13), (13, 14), (14, 15), (15, 16),
        (13, 17), (17, 18), (18, 19), (19, 20),
        (0, 17)
    ]
def find_hand_landmarks(self, frame, draw=True):
    h, w, _ = frame.shape
    landmarks_list = []
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    if self.use_legacy_solutions:
        results = self.detector.process(rgb_frame)
        if results.multi_hand_landmarks:
            first_hand = results.multi_hand_landmarks[0]
            for idx, lm in enumerate(first_hand.landmark):
                cx, cy = int(lm.x * w), int(lm.y * h)
                landmarks_list.append((idx, cx, cy, lm.z))
            if draw:
                self.mp_draw.draw_landmarks(
                    frame,
                    first_hand,
                    self.mp_hands.HAND_CONNECTIONS
                )
```

```
        else:
                mp_image = mp.Image(
                image_format=mp.ImageFormat.SRGB,
                data=rgb_frame
        )
        results = self.detector.detect(mp_image)
        if results.hand_landmarks:
            first_hand = results.hand_landmarks[0]
            for idx, lm in enumerate(first_hand):
                cx, cy = int(lm.x * w), int(lm.y * h)
                landmarks_list.append((idx, cx, cy, lm.z))
            if draw:
                self._draw_landmarks(frame, landmarks_list)
    return landmarks_list, frame
def _draw_landmarks(self, frame, landmarks):
    for p1, p2 in self.CONNECTIONS:
        x1, y1 = landmarks[p1][1], landmarks[p1][2]
        x2, y2 = landmarks[p2][1], landmarks[p2][2]
        cv2.line(
            frame, (x1, y1), (x2, y2),
            (200, 200, 200), 2, cv2.LINE_AA
        )
    for idx, cx, cy, _ in landmarks:
        if idx in (8, 12):
            cv2.circle(frame, (cx, cy), 6, (0, 255, 255), -1)
            cv2.circle(frame, (cx, cy), 8, (255, 255, 255), 1)
        else:
            cv2.circle(frame, (cx, cy), 4, (0, 180, 255), -1
```

6.1 System Implementation and outputs

The implementation of the Real-Time Air Drawing Application is configured using Python, OpenCV, MediaPipe, and NumPy. The system captures webcam video, detects hand landmarks, and converts index finger movements into drawing actions on a virtual canvas.

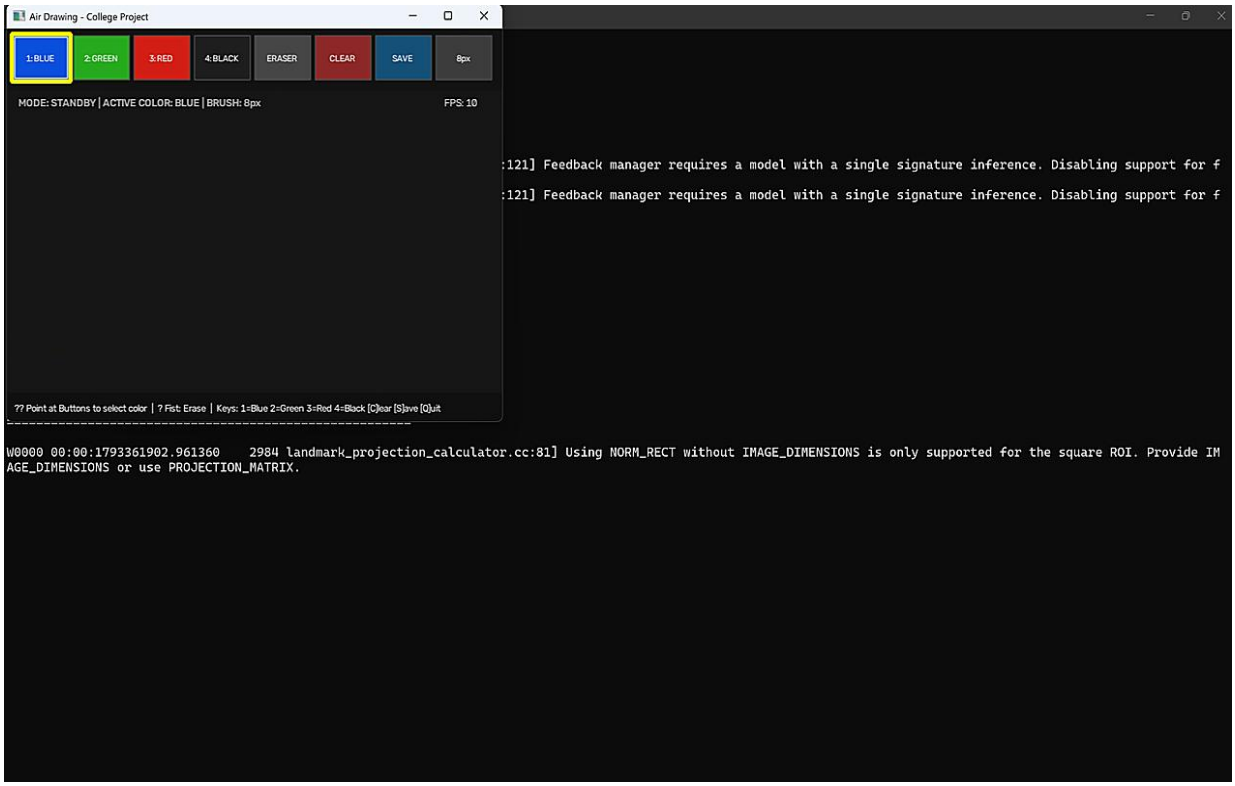


Fig 6.1.1 Virtual Canvas Manager Implementation

The application is executed by running the main Python program. OpenCV captures the webcam feed, while MediaPipe detects and tracks hand landmarks. The detected finger coordinates are used to generate strokes on the virtual canvas. Users can clear the canvas and save their drawings using the configured controls.

The output demonstrates touchless drawing through real-time hand gesture recognition. The system can be extended with color selection, erasing, undo, redo, and Agentic AI-based

CHAPTER-7

RESULTS AND DISCUSSION

7.1 Results

The Real-Time Air Drawing Application was designed to provide touchless drawing using hand gestures captured through a webcam. The system uses OpenCV for video processing and MediaPipe for hand landmark detection. The detected index finger movements are converted into strokes on a virtual canvas, allowing users to draw without physical contact with the computer.

Overall, the Real-Time Air Drawing Application provides a foundation for contactless human-computer interaction in educational demonstrations, presentations, and creative activities. Further evaluation using measured response times, gesture recognition accuracy, and task completion rates would help quantify the application's performance and reliability.

The application successfully demonstrates the conversion of hand movements into digital drawing actions. By tracking the index finger in real time, users can create drawings on the virtual canvas with smooth and continuous movements. The system provides an interactive drawing experience and reduces the need for traditional input devices such as a mouse or touchscreen.

The application also demonstrates how computer vision techniques can improve human-computer interaction. Its performance depends on factors such as camera quality, lighting conditions, and hand visibility. The project shows the practical use of Python, OpenCV, MediaPipe, and NumPy in developing a real-time touchless drawing system.

7.2 Discussion

The Real-Time Air Drawing Application was designed to provide touchless drawing through real-time hand gesture recognition. The system combines OpenCV for webcam video processing and MediaPipe for hand landmark detection to convert finger movements into drawing actions on a virtual canvas.

Overall, the application demonstrates the potential of combining computer vision, hand tracking, and intelligent action selection for natural human-computer interaction. Further evaluation using actual measurements of gesture recognition accuracy, drawing response time, and task completion rate would help determine its effectiveness and identify areas for improvement.

The application also demonstrates how computer vision techniques can improve human-computer interaction. Its performance depends on factors such as camera quality, lighting conditions, and hand visibility. The project shows the practical use of Python, OpenCV, MediaPipe, and NumPy in developing a real-time touchless drawing system.

Overall, the application demonstrates the potential of combining computer vision, hand tracking, and intelligent action selection for natural human-computer interaction. Further evaluation using actual measurements of gesture recognition accuracy, drawing response time, and task completion rate would help determine its effectiveness and identify areas for improvement

7.3 Existing system vs Proposed System

The existing-system comparison refers to traditional drawing methods that use a mouse, keyboard, or touchscreen. The proposed system uses computer vision and hand gesture recognition to enable touchless drawing through a webcam.

Table 7.3 Existing system and proposed-system comparison

Aspect	Existing System	Proposed System
Input method	Mouse, keyboard, or touchscreen	Hand gestures using webcam
Drawing control	Manual input	Index finger movement
Technology	Conventional drawing tools	OpenCV and MediaPipe
Interaction	Physical contact required	Touchless interaction
Intelligent features	Basic drawing operations	Agentic AI-based action selection

The proposed Air Drawing Application provides a touchless drawing experience using computer vision and hand gesture recognition. It reduces dependence on physical input devices and supports natural interaction. The integration of Agentic AI can further improve gesture-based action selection, making the system useful for education, presentations, and creative activities.

CHAPTER-8

CONCLUSION

8.1 Project Conclusion

The Real-Time Air Drawing Application provides a touchless drawing system that allows users to draw on a virtual canvas using hand gestures. The application combines OpenCV for real-time video processing and MediaPipe for hand landmark detection to convert finger movements into drawing actions.

The system supports interactive drawing and features such as color selection, erasing, clearing, and saving drawings, depending on the implemented functionality. The integration of Agentic AI provides a foundation for interpreting gesture events and selecting appropriate actions, improving the potential for intelligent human-computer interaction.

The project demonstrates how computer vision, hand tracking, and artificial intelligence can be combined to create a simple and natural drawing interface without requiring direct physical contact. It can be useful in educational environments, presentations, demonstrations, and creative activities. In the future, the application can be enhanced with more accurate gesture recognition, improved drawing smoothness, voice commands, and additional AI-powered features.

Future Scope

The Real-Time Air Drawing Application has significant potential for future improvements. The system can be enhanced by incorporating advanced hand gesture recognition techniques to improve drawing accuracy and reduce unwanted strokes. Additional features such as adjustable brush sizes, multiple colors, shape drawing, text insertion, and advanced editing tools can make the application more versatile and user-friendly.

In the future, voice commands can be integrated to allow users to control drawing operations without physical contact. Multi-hand tracking can enable more complex interactions, while cloud storage can help users save and share their drawings easily. The integration of Agentic AI can further improve the system by interpreting user intentions and selecting suitable drawing actions automatically.

The application can also be extended for educational purposes, online presentations, digital demonstrations, and creative activities. With improvements in accuracy, responsiveness, and intelligent interaction, the system can become a more effective and accessible platform for touchless human-computer interaction.

REFERENCES

[1] Zhang, F.; Bazarevsky, V.; Vakunov, A.; Tkachenka, A.; Sung, G.; Chang, C.-L.; Grundmann, M. MediaPipe Hands: On-device Real-time Hand Tracking. 2020. https://doi.org/10.48550/arXiv.2006.10214

[2] Bradski, G. The OpenCV Library. Dr. Dobb's Journal of Software Tools (2000). https://opencv.org/

[3] Kumar, R. N.; Vaishnavi, M.; Gayatri, K.; Prashanthi, V.; Supriya, M. Air Writing Recognition using MediaPipe and OpenCV. Smart Innovation, Systems and Technologies, 302 (2022). https://doi.org/10.1007/978-981-19-2541-2_35

[4] Sung, G.; Sokal, K.; Uboweja, E.; Bazarevsky, V.; Baccash, J.; Bazavan, E. G.; Chang, C.-L.; Grundmann, M. On-device Real-time Hand Gesture Recognition. arXiv preprint arXiv:2111.00038 (2021). https://arxiv.org/abs/2111.00038

[5] Uboweja, E.; Tian, D.; Wang, Q.; Kuo, Y.-C.; Zou, J.; Lu, W.; Sung, G.; Grundmann, M. On-device Real-time Custom Hand Gesture Recognition. arXiv preprint arXiv:2309.10858 (2023). https://arxiv.org/abs/2309.10858

[6] Subramaniam, R. R.; Nethranand, M.; Srija, M. Virtual Painter Using Hand Gesture Recognition with OpenCV and MediaPipe. International Conference on Computational Robotics, Testing and Engineering Evaluation (ICCRTEE) (2025). https://doi.org/10.1109/ICCRTEE64519.2025.11053081

[7] Iffath, A.; Ramabai, M.; Kotaiahswamy; Tejikhaa, T. Air Canvas Using OpenCV and MediaPipe. Journal of Emerging Technologies and Innovative Research, 12, 5, e666–e673 (2025). https://www.jetir.org/view?paper=JETIR2505551

JETIR

REFERENCES CONTINUED

[8] Yasar, P.; Mehvish, S. Air Canvas Using Python-OpenCV. International Journal for Research Trends and Modernization, 5, 5, 47–53 (2025). https://doi.org/10.59256/ijrtmr.20250505009

[9] Zhang, F.; Bazarevsky, V.; Vakunov, A.; Tkachenka, A.; Sung, G.; Chang, C.-L.; Grundmann, M. On-Device, Real-Time Hand Tracking with MediaPipe. Google Research Blog (2019). https://research.google/blog/on-device-real-time-hand-tracking-with-mediapipe/

[10] Virtual Air Drawing Using Computer Vision and Real-Time Captions. International Journal for Research in Applied Science and Engineering Technology (2026). https://doi.org/10.22214/ijraset.2026.76769

[11] Sridevi, M.; Prasad, D. S. AirCanvas: A Real-Time Drawing Application with Hand Gesture Recognition and Automated Shape Beautification. Procedia Computer Science, 283, 1202–1209 (2026). https://doi.org/10.1016/j.procs.2026.06.199

[12] Jaivin, S.; Aswani, S. Real-Time Human-Computer Interaction: A Low-Latency Virtual Painter System Using MediaPipe and OpenCV. Journal of Advance and Future Research, 4, 2, 489–498 (2026). https://rjwave.org/JAAFR/papers/JAAFR2602279

[13] Devika, G. Y. Virtual Touch System Using MediaPipe and OpenCV for Gesture-Based Human-Computer Interaction. International Journal of Science and Modern Technology (2026). https://doi.org/10.55041/ijsmt.v2i5.592

[14] OpenCV Team. OpenCV Documentation: Image Processing and Computer Vision Library. https://docs.opencv.org/